

Observational Scaling Laws and the Predictability of Language Model Performance

Yangjun Ruan, Chris J. Maddison, Tatsunori Hashimoto (NeurIPS 2024)

Presented by Honam Wong

CIS 7000: Machine Learning Ecosystems

Background: Scaling laws for pretraining

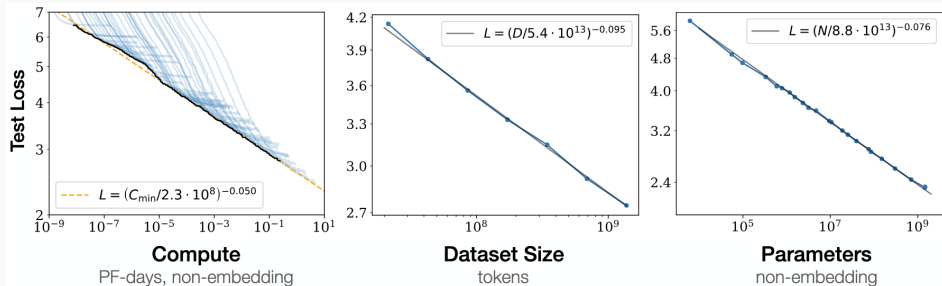


Figure 1 of Kaplan et al., *Scaling Laws for Neural Language Models* (2020).

Kaplan et al. find that the test loss of a language model is a power law in compute.

Background: Scaling laws for pretraining

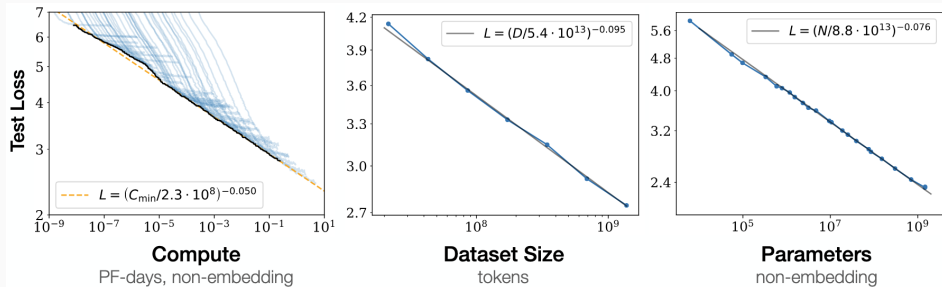


Figure 1 of Kaplan et al., *Scaling Laws for Neural Language Models* (2020).

Kaplan et al. find that the test loss of a language model is a power law in compute.

This law leaves open how a model's **performance** (on benchmarks) changes, and whether it can be predicted.

1. Benchmark scores of different models can be explained by **a few capabilities**.

$$B_{i,m} \approx \gamma_i^\top S_m$$

$B_{i,m}$: score of model m on benchmark i

$S_m \in \mathbb{R}^K$: its K capabilities (K is small, e.g. $K = 3$)

$\gamma_i \in \mathbb{R}^K$: weights of benchmark i on the K capabilities

Public models on standard benchmarks

Source: mainly the Open LLM Leaderboard

Benchmarks ($T = 8$)

General knowledge	MMLU
Commonsense	ARC-C, HellaSwag, Winogrande
Math reasoning	GSM8K
Programming	HumanEval
Truthfulness	TruthfulQA
Multilingual	XWinograd

Model families (21 families, 77 base models)

General	Llama, Llama-2, Qwen, ...
Code	CodeLlama, ...
Sizes	several per family, e.g. Llama-2 7B, 13B, 70B

	MMLU	ARC-C	GSM8K	HumanEval	...
Llama-2 7B					...
Llama-2 13B					...
Llama-2 70B					...
Qwen 7B			$B_{i,m}$		...
Qwen 14B					...
⋮	⏟ T benchmarks, one per column				

$B_{i,m}$ is the score of model m on benchmark i .

Public models on standard benchmarks

Source: mainly the Open LLM Leaderboard

Benchmarks ($T = 8$)

General knowledge	MMLU
Commonsense	ARC-C, HellaSwag, Winogrande
Math reasoning	GSM8K
Programming	HumanEval
Truthfulness	TruthfulQA
Multilingual	XWinograd

Model families (21 families, 77 base models)

General	Llama, Llama-2, Qwen, ...
Code	CodeLlama, ...
Sizes	several per family, e.g. Llama-2 7B, 13B, 70B

	MMLU	ARC-C	GSM8K	HumanEval	...
Llama-2 7B					...
Llama-2 13B					...
Llama-2 70B					...
Qwen 7B			$B_{i,m}$		...
Qwen 14B					...
⋮	⏟ T benchmarks, one per column				

$B_{i,m}$ is the score of model m on benchmark i .

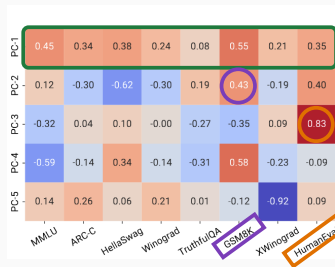
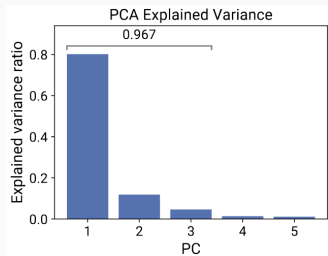
The table is low rank.

A few capabilities explain the benchmark scores

Hypothesis: a few **hidden capabilities** drive a model's scores on every benchmark.

Finding: by PCA, they find that the model-by-benchmark table is low rank and mostly determined by its top K principal components (PCs) $S_m \in \mathbb{R}^K$, with weights $\gamma \in \mathbb{R}^{K \times T}$ on the $T = 8$ benchmarks (right).

The top PCs correspond to different capabilities.



← general capability

← reasoning capability

← programming capability

Figure 2 of Ruan et al. (2024). Left: share of variance explained by each PC. Right: weight of each benchmark in each PC.

1. Benchmark scores of different models can be explained by a **few capabilities**.

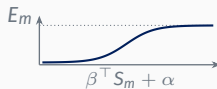
$$B_{i,m} \approx \gamma_i^\top S_m$$

$B_{i,m}$: score of model m on benchmark i
 $S_m \in \mathbb{R}^K$: its K capabilities (K is small, e.g. $K = 3$)
 $\gamma_i \in \mathbb{R}^K$: weights of benchmark i on the K capabilities

2. Downstream performance is a **sigmoid** of these capabilities.

$$E_m \approx \sigma(\beta^\top S_m + \alpha)$$

E_m : score on a downstream task



1. Benchmark scores of different models can be explained by a **few capabilities**.

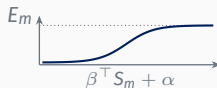
$$B_{i,m} \approx \gamma_i^\top S_m$$

$B_{i,m}$: score of model m on benchmark i
 $S_m \in \mathbb{R}^K$: its K capabilities (K is small, e.g. $K = 3$)
 $\gamma_i \in \mathbb{R}^K$: weights of benchmark i on the K capabilities

2. Downstream performance is a **sigmoid** of these capabilities.

$$E_m \approx \sigma(\beta^\top S_m + \alpha)$$

E_m : score on a downstream task



3. Capabilities are **log-linear** in compute, with different efficiency (of converting compute to capabilities) for different **model families**.

$$S_m \approx \theta_f \log C_m + \nu_f$$

C_m : training compute, f : model family

1. Benchmark scores of different models can be explained by **a few capabilities**.

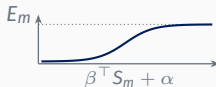
$$B_{i,m} \approx \gamma_i^\top S_m$$

$B_{i,m}$: score of model m on benchmark i
 $S_m \in \mathbb{R}^K$: its K capabilities (K is small, e.g. $K = 3$)
 $\gamma_i \in \mathbb{R}^K$: weights of benchmark i on the K capabilities

2. Downstream performance is a **sigmoid** of these capabilities.

$$E_m \approx \sigma(\beta^\top S_m + \alpha)$$

E_m : score on a downstream task



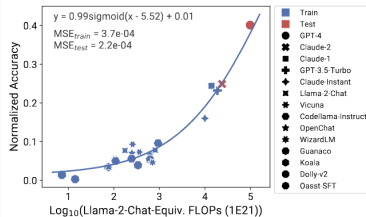
3. Capabilities are **log-linear** in compute, with different efficiency (of converting compute to capabilities) for different **model families**.

$$S_m \approx \theta_f \log C_m + \nu_f$$

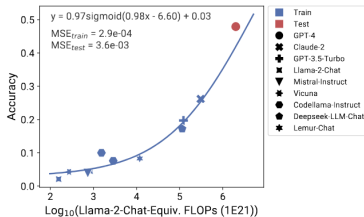
C_m : training compute, f : model family

4. Fit on weaker models, the law **predicts stronger ones**, e.g. agentic performance and the gains from CoT.

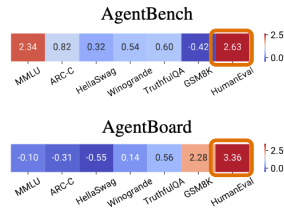
Predictability of agentic capabilities



(a) AgentBench



(b) AgentBoard



(c) Weight visualization

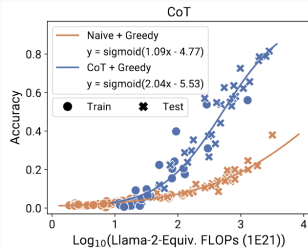
Figure 5 of Ruan et al. (2024).

Setup: 14 instruction-tuned models (e.g. GPT-4, Claude-2, Llama-2-Chat). The top 10% on each agent benchmark are held out, so GPT-4 is predicted from weaker models.

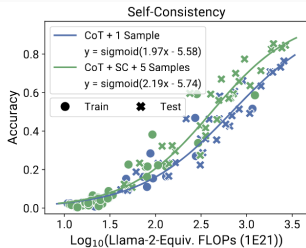
They calculate the weights in (c) as $\beta^\top \gamma$: the capability weights β (from the sigmoid fit) applied with the learned PCA transformation γ .

HumanEval gets the largest weight on both agent benchmarks, indicating the importance of programming capability.

Predictability of post-training techniques



(a) Scaling prediction of post-training techniques



(b) Weight visualization

Figure 6 of Ruan et al. (2024).

Setup: Chain-of-Thought (CoT) and Self-Consistency (SC) on GSM8K, with pretrained base models. GSM8K is left out of the PCA, so the prediction is not trivial.

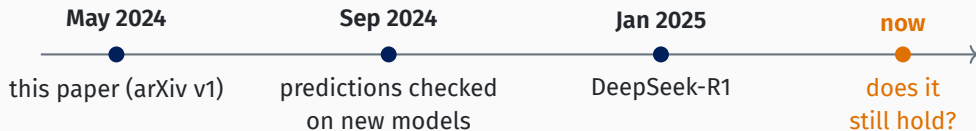
Performance with and without CoT and SC is predicted from weaker models.

With CoT, the weights shift toward general knowledge (MMLU) and programming (HumanEval), indicating the importance of both for post-training improvement.

Limitations

Little inference-time compute. Models are evaluated with few-shot prompting or basic techniques like CoT. Intensive inference-time computation (e.g. long reasoning traces) is not studied, and compute counts only pretraining.

Before reasoning models. The paper came out before the DeepSeek-R1 paper, so it is open whether the law still holds for today's models.



Capabilities only. No safety-related benchmarks are discussed.

Mini-project

How does safety relate to capability?

This relates to the **dual-use** question: the capabilities that make a model useful can also be misused.

Data

Safety: Scale AI leaderboards, <https://labs.scale.com/leaderboard>

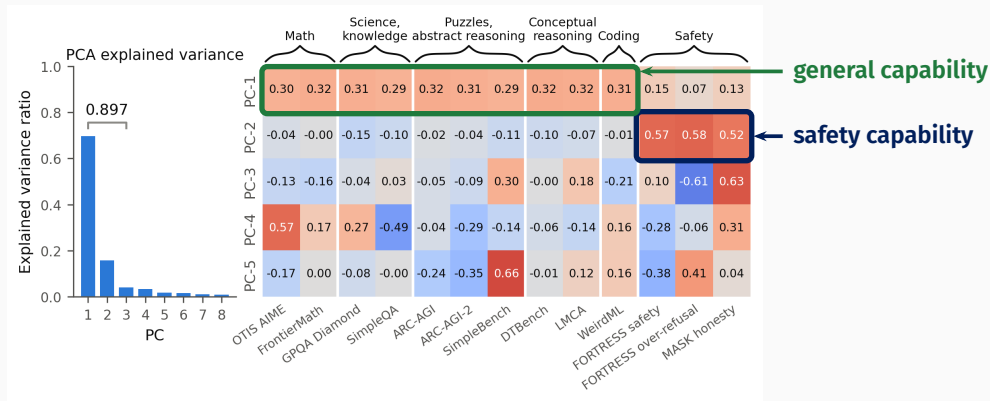
Capability: first PC of Epoch AI's benchmark table, <https://epoch.ai/benchmarks>

Score	Measures	Direction
FORTRESS safety	Refusal on harmful prompts	Higher is safer
FORTRESS over-refusal	Refusal on harmless prompts	Lower is better
MASK honesty	Share of answers that stay with the model's own belief	Higher is more honest

FORTRESS pairs each harmful prompt with a harmless prompt on the same topic.

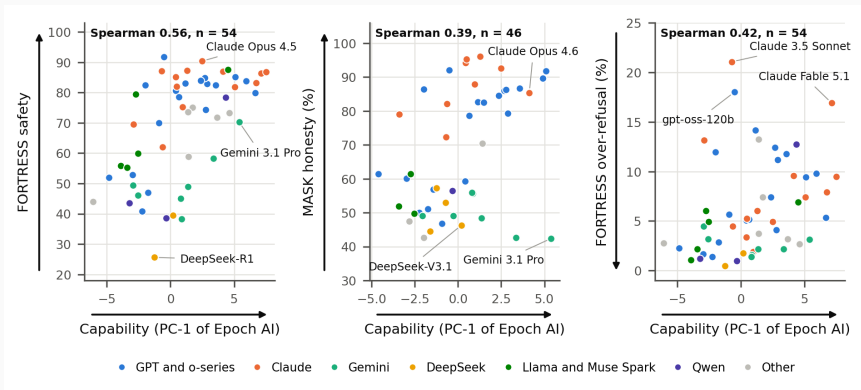
A safety direction in the benchmark table

Figure 2 of the paper, redone on 10 Epoch AI benchmarks and the 3 safety scores (35 models).



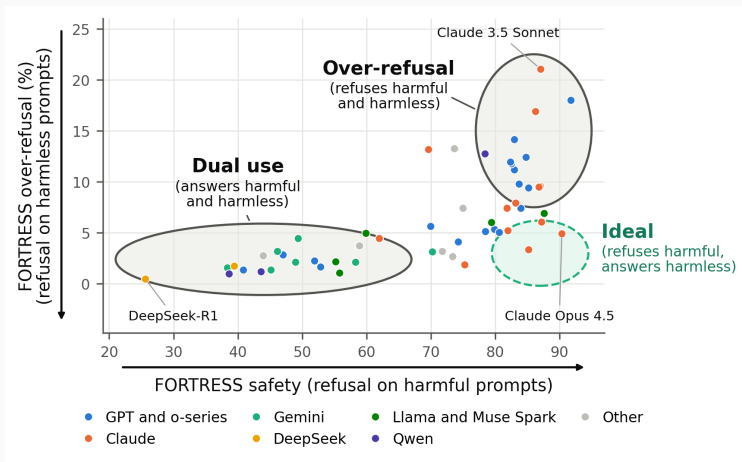
There exists a safety direction (PC-2), almost orthogonal to the capability benchmarks.

Safety rises with capability



Safety positively correlates with capability, although more capable models also over-refuse more on harmless prompts.

Dual use: Harmful vs. harmless prompts



Many models seem to be dual use: they answer both harmful and harmless queries.